

計算機通論期末レポート

電気通信大学 I 類

2311207 宮川一步

2024 年 8 月 1 日作成

1 課題 1：プログラム格納（内臓）型計算機の動作原理を、ポンチ絵等を使いながら説明せよ。

プログラム格納方式とは、メモリ（主記憶）に配置されたアドレスに対してバス内のアドレス線を通して命令を指定し、解釈・実行を行うコンピュータ・アーキテクチャの方式の一つである。プログラム格納方式を採用した計算機は、主にメモリ・プロセッサ・入出力およびそれら 3 要素を結合するバスの 4 要素から構成され、各要素の対応関係は以下の図のようにになっている。

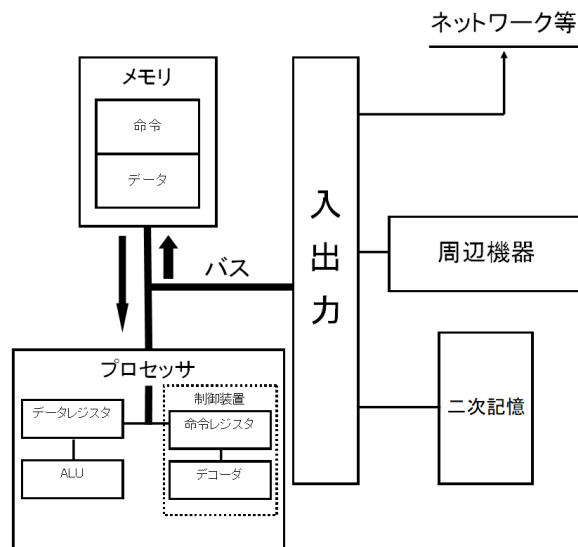


図 1: プログラム格納型計算機の概形

この計算機の構造について整理すると、まずメモリ（一次記憶）には、実行する命令と処理するデータが帯状になって格納されており、入出力には様々な周辺機器やネットワーク等が存在している。

これらの構成要素とプロセッサ（中央処理装置）は、バス内部に存在するアドレス線・データ線・コントロール線を通してデータのやり取りを行っており、プロセッサ内部の命令レジスタおよびプログラムカウンタに沿って命令の実行を行い、必要に応じてプロセッサ側から各要素の制御を行う。

プロセッサの動作は、命令取得・解釈・実行の順に構成され、この 3 つの工程をプログラムカウンタに沿って稼働することで、正常にプログラムが実行される。

1. 命令取得 (Fetch)：アドレス線を通してメモリ内の番地を指定し、番地内に格納されている命令をデータ線を通して取得する。
2. 解釈 (Decode)：プロセッサ内部のデコーダを通して取り出した命令を解釈し、実行に必要な処理を決定する。
3. 実行 (Execute)：指定された処理に基づいて命令の実行を行い、コントロール線を通してメモリ内のデータや周辺機器、ALU(算術演算装置)の制御を行い、必要に応じて実行結果の書き込みを行う。

2 課題2：nビットの符号付き数（2の補数表示）の大小比較する比較器を構成せよ。

本項では、課題で与えられていたC言語による比較器のプログラムを基に、符号付数の大小比較を行う比較器を構成した。

Listing 1: 比較器のプログラム

```
1 #include <stdio.h>
2 int n=0;
3 int compare_us(unsigned x[n-1], unsigned y[n-1])
4 {
5     int i;
6     for(i = 1; i < n; i++){
7         if(x[i] < y[i])
8             return 1;
9         else if(y[i] < x[i])
10            return 0;
11    }
12    return 0;
13 }
14 int compare_s(int x[n-1], int y[n-1]) {
15     if(x[0] > y[0])
16         return 1;
17     else if(y[0] > x[0])
18         return 0;
19     else
20         compare_us(x,y);
21 }
22 int main(void){
23     int s,t;
24     int x[] = {1,1,0,1,1,1};
25     int y[] = {0,0,0,1,1,0,1,0};
26     s = sizeof x / sizeof x[0];
27     t = sizeof y / sizeof y[0];
28     if(s == t){n = s;printf("%d\n",compare_s(x,y));}
29     else if(s > t){
30         n = s;int z[s];
31         int st = s - t;
32         for(int j = 0; j < st; j++){z[j] = y[0];}
33         for(int j = 0; j < t; j++){z[j+st] = y[j];}
34         printf("%d\n",compare_s(x,z));
35     }
36     else{
37         n = t;int z2[t];
38         int ts = t - s;
39         for(int j = 0; j < ts; j++){z2[j] = x[0];}
40         for(int j = 0; j < s; j++){z2[j+ts] = x[j];}
41         printf("%d\n",compare_s(z2,y));
42     }
43     return 0;
44 }
```

まず、main 関数では、配列 x, y に n ビットの符号付き数を入力する。入力から x, y の各ビット数を算出し、もしビット数が揃っていない場合、符号拡張を用いてビット数を揃えるようにしている。

ビット数をそろえたのち、まずは符号比較を行う素子、`compare_s` に x および y を入力し、符号ビットの比較を行う。 y が大きい場合 (y の符号ビットが 0 で x の符号ビットが 1 の場合)、1 を返し、 x が大きい場合 0 を返す。

次に、2 値とも符号が揃っていた場合、符号なし数の比較を行う素子、`compare_us` に値を入力し、上位ビットから順に比較を行い、 y が大きい場合 1 を返し、その他の場合に 0 を返す。

こうすることで、符号なし数の比較素子を組み合わせて、符号あり数の大小比較を行う比較器を構成することができた。

3 課題 3 : MIPS アーキテクチャの ISA を、 レジスタ構成を図示しながらまとめよ。

MIPS アーキテクチャは、表 1 における 32 本の汎用レジスタにより構成され、以下の図 1 のようなレジスタ構成を持っている。

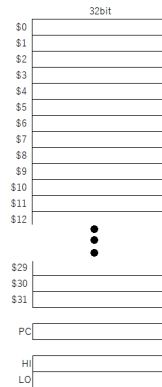


図 2: MIPS のレジスタ構成

表 1: 汎用レジスタの名前・機能

レジスタ名	機能名	特徴・用途
\$0	\$zero	常に 0 の値が読みだされる
\$1	\$at	アセンブラ用の一次レジスタ
\$2	\$v0	関数の返り値を格納する領域、syscall の動作決定にも使用。
\$3	\$v1	関数の返り値 (64bit)
\$4	\$a0	関数の 1 番目の引数
\$5	\$a1	関数の 2 番目の引数
\$6	\$a2	関数の 3 番目の引数
\$7	\$a3	関数の 4 番目の引数
\$8 ~ 15	\$t0 ~ t7	作業用の自由領域
\$16 ~ 23	\$s0 ~ s7	作業用の自由領域。値が保持される。
\$24,25	\$t8,t9	位置独立コードの間接ジャンプ用領域。アドレスが保存される
\$26,27	\$k0,k1	OS のカーネル用領域
\$28	\$gp	グローバル変数へのベースポインタ
\$29	\$sp	スタックポインタ
\$30	\$fp	フレームポインタ
\$31	\$ra	サブルーチンの戻り値を保存する領域

それぞれのレジスタごとに 4 バイト : 32bit のデータ領域を持っており、メモリ上のデータをレジスタにロード・ストアすることで、メモリ上のデータを処理している。レジスタを使用することで、直接アドレスを指定

することができないメモリ上のデータを処理し、書き込むことができるようになっている。また、MIPS アーキテクチャの命令は、32 ビットの固定長で処理され、以下の 3 種類のフォーマットを持っている。

3.1 R 形式

表 2: R 形式

op(6bit)	rs(5bit)	rt(5bit)	rd(5bit)	shamt(5bit)	funct(6bit)
----------	----------	----------	----------	-------------	-------------

演算用命令。funct の値により演算の内容を区別している。加減算などの処理や論理演算がこれに当たる。

3.2 I 形式

表 3: I 形式

op(6bit)	rs(5bit)	rt(5bit)	Imm(16bit)
----------	----------	----------	------------

主にデータ転送や即値演算に使用される形式。Imm に 16bit の即値を入れることができる。addi(即値加算) や lw(ワード形式での読み込み) などがこれに当たる。

3.3 J 形式

表 4: J 形式

op(6bit)	address(26bit)
----------	----------------

アドレスジャンプ用の命令形式。j などの命令が J 形式に当たる。

4 課題4：プログラム言語の処理方式であるインタプリタとコンパイラについて説明せよ。そして、プロセッサはどちらの一種であるかを理由と共に説明せよ。

4.1 インタプリタ

インタプリタとは、ある特定のプログラミング言語で記述されたプログラムをその文法に沿って逐次解釈を行い、直接命令を動作させる処理手法である。インタプリタでは、逐次解釈を行ったのちに、都度命令の実行を行うことから、繰り返し等の処理において同じ命令が重複して解釈される可能性があり、コンパイラと比較すると実行速度は遅くなりやすい。対して開発面では、コードの各命令を逐次処理していくことから、稼働結果に基づいてリアルタイムでの対応がしやすく、変更の反映も早いことから、デバッグ及び開発が容易であるという特徴がある。

4.2 コンパイラ

コンパイラとは、ある言語で記述されたプログラムコードを一度に解釈し、実行可能な形式も含めた他の言語による記述に変換する処理手法である。コンパイラにおいては、解釈・実行の内、解釈のみが行われるようになっており、直接実行可能にするというよりは、機械語等の実行可能な形式も含めた、別の言語への変換を行うものである。特徴としては、一度コンパイラを通したコードは追加の解釈が必要ないため、重複解釈などを省くことができ、命令を高速に実行することができる。対して、プログラムコードに変更が生じた場合、再度全体をコンパイラに通す必要があることから、開発スパンが長期化しやすいという特徴がある。

4.3 プロセッサの分類

プロセッサとは、メモリから機械語の命令を読み出し、解読した動作を実行していく計算ユニットである。プロセッサの動作原理は、メモリからプログラムカウンタに基づいて逐次命令を読み出し、解釈および実行を行うというものであり、逐次命令を解釈し実行に移すという動作行程から、プロセッサはインタプリタの1種であるといえる。ただし、インタプリタはソフトウェアを主体とした処理手法の1つであるのに対して、プロセッサは処理を行うハードウェアであることから、処理動作的には同種とみなせるものの、厳密には処理を行う操作レベルが異なっている。

5 課題5：フィボナッチ数を計算するC言語のプログラムをMIPSに沿って解く。

5.1 5-1：C言語によるフィボナッチ数を計算するプログラムを稼働させ、結果を示す

・実行結果

Listing 2: K_05.c result

```
1 m2311207@orange01:~/KEISANKI$ gcc K_05.c
2 m2311207@orange01:~/KEISANKI$ ./a.out
3 2,1
4 3,2
5 4,3
6 5,5
7 6,8
8 7,13
9 8,21
10 9,34
11 10,55
12 11,89
13 12,144
14 13,233
15 14,377
16 15,610
17 16,987
18 17,1597
19 18,2584
20 19,4181
21 20,6765
22 21,10946
23 22,17711
24 23,28657
25 24,46368
26 25,75025
27 26,121393
28 27,196418
29 28,317811
30 29,514229
```

・説明本プログラムでは、フィボナッチ数の計算を再帰関数を用いて求めるものであり、出力はカンマを挟んで、各回における*i*の値と求めたフィボナッチ数の値を出力するようになっている。動的計画法を採用していないことから、計算時間は*i*の値が大きくなるにつれて増加していく。

5.2 5-2：フィボナッチ数を計算するプログラムをMIPSアセンブリ言語に変換し、稼働結果を調べる。

・プログラム

Listing 3: K_06.asm

```
1 .data
2 newline: .asciiz "\n"
```

```

3 comma: .asciiz ","
4
5     .text
6     .globl main
7     .ent main
8
9 main:
10     subu $sp, $sp, 20
11     sw $ra, 12($sp)
12     li $s0, 2
13     li $s1, 0
14 loop:
15     move $a0, $s0
16     jal fib
17     move $s1, $v0
18
19     move $a0, $s0
20     li $v0, 1
21     syscall
22
23     la $a0, comma
24     li $v0, 4
25     syscall
26
27     move $a0, $s1
28     li $v0, 1
29     syscall
30
31     la $a0, newline
32     li $v0, 4
33     syscall
34
35
36     add $s0, $s0, 1
37     sltu $t0, $s0, 30
38     bgtz $t0, loop
39
40     move $v0, $0
41     lw $ra, 12($sp)
42     addu $sp, $sp, 20
43     jr $ra
44
45 fib:
46     subu $sp, $sp, 16
47     sw $ra, 4($sp)
48     li $t1, 1
49     beq $a0, $0, return0
50     beq $a0, $t1, return1
51
52     sw $a0, 8($sp)

```



```

53      subu $a0, $a0, 1
54      jal fib
55
56      sw $v0, 12($sp)
57      lw $a0, 8($sp)
58      subu $a0, $a0, 2
59      jal fib
60
61      lw $t0, 12($sp)
62      lw $ra, 4($sp)
63      add $v0, $v0, $t0
64      addu $sp, $sp, 16
65      jr $ra
66
67 return0:
68      li $v0, 0
69      addu $sp, $sp, 16
70      jr $ra
71
72 return1:
73      li $v0, 1
74      addu $sp, $sp, 16
75      jr $ra
76 .end

```

• 稼働結果

Listing 4: K_06.asm result

```

1 (spim) load "K_06.asm"
2 (spim) run
3 2,1
4 3,2
5 4,3
6 5,5
7 6,8
8 7,13
9 8,21
10 9,34
11 10,55
12 11,89
13 12,144
14 13,233
15 14,377
16 15,610
17 16,987
18 17,1597
19 18,2584
20 19,4181
21 20,6765
22 21,10946

```

```
23 22,17711
24 23,28657
25 24,46368
26 25,75025
27 26,121393
28 27,196418
29 28,317811
30 29,514229
```

・説明 5-1 で使用したフィボナッチ数を求めるプログラムを MIPS を用いて組みなおし、実行を行った。以上のように、出力結果が 5-1 における出力と一致していることが確認できる。

5.3 5-3 : fib() 関数を呼び出した回数を求められるようにプログラムを改良し、計算結果と呼び出し回数の関係を調べる。

・改良したプログラム

Listing 5: K_06.asm

```
1 #include <stdio.h>
2
3 int n = 0;
4 int m;
5
6 int fib(int x) {
7     m++;
8     if (x == 0) {return 0;}
9     else if (x == 1) {return 1;}
10    else {return fib(x-1) + fib(x-2);}
11 }
12
13 int main() {
14     int r, i;
15     for (i = 2; i < 30; i++) {
16         m = 0;
17         r = fib(i);
18         n = n + m;
19         printf("%d,%d,%d,%d\n", i, r, m, n);
20     }
21 }
```

・出力結果

Listing 6: K_06.asm result

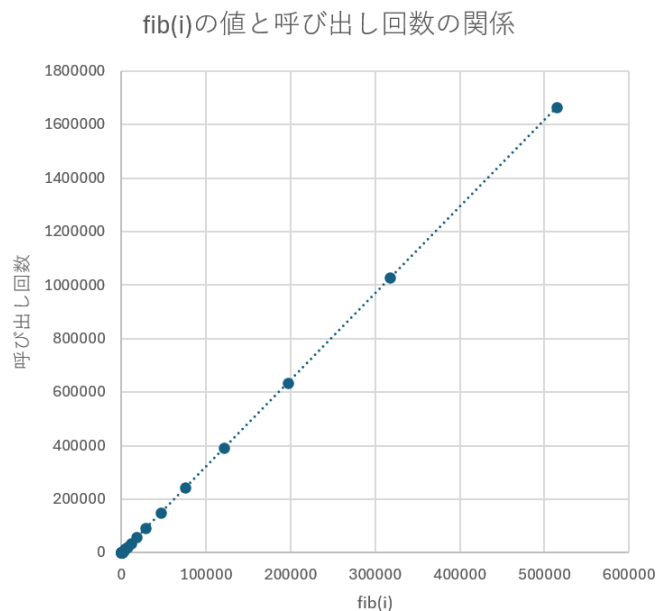
```
1 2,1,3,3
2 3,2,5,8
3 4,3,9,17
4 5,5,15,32
5 6,8,25,57
6 7,13,41,98
```

```

7 8,21,67,165
8 9,34,109,274
9 10,55,177,451
10 11,89,287,738
11 12,144,465,1203
12 13,233,753,1956
13 14,377,1219,3175
14 15,610,1973,5148
15 16,987,3193,8341
16 17,1597,5167,13508
17 18,2584,8361,21869
18 19,4181,13529,35398
19 20,6765,21891,57289
20 21,10946,35421,92710
21 22,17711,57313,150023
22 23,28657,92735,242758
23 24,46368,150049,392807
24 25,75025,242785,635592
25 26,121393,392835,1028427
26 27,196418,635621,1664048
27 28,317811,1028457,2692505
28 29,514229,1664079,4356584

```

与えられたプログラムを改良し、 i および $\text{fib}(i)$ の出力に加えて、各フィボナッチ数の計算過程における $\text{fib}()$ 関数の呼び出し回数と、全体での呼び出し回数の総計を出力できるようにした。出力結果から、各フィボナッチ数に対応した $\text{fib}()$ の呼び出し回数は、呼び出し回数 = 1 つ前の呼び出し回数 + 2 つ前の呼び出し回数 + 1 によって求めることができ、2 値の相関関係は以下のように比例関係が成り立っていると考えられる。



この理由は、fib(i) を求める過程において、i-1 および i-2 における fib() を求める工程が組み込まれており、動的計画法を採用していないことから、再帰呼び出しの重複が生じている。

よって、fib(i) における fib 関数の呼び出し回数および実行時間は fib(i) の値が増加すると同時に指数関数的に増加していくと考えられ、呼び出し回数は 2^{k-1} 回、計算量オーダーは $O(n^2)$ となることから、大きい i になるほどこのアルゴリズムによる計算手法は不適切になっていくと考えることができる。

改善策としては動的計画法を採用が挙げられ、無駄な再帰呼び出しの除去ができることから、稼働効率を改善することができる。

6 課題6：アウトオブオーダー実行のスーパースカラを構成する技術としてスコアボーディングとトマスロのアルゴリズムがある。両者について調べてまとめよ。

6.1 スコアボーディング

スコアボーディングとは、命令同士の依存関係を管理しながら複数命令の実行スケジューリングを行う技術の1つである。

スコアボーディング方式では、単純にプログラムに沿った順に命令を実行していくのではなく、実行時間が比較的長い命令を実行している最中に、依存関係のない命令を並列的に実行してしまうようになっており、複数のパイプラインを効率的に実行に移していくことができる。

スコアボードは、命令における以下の3項目のステータスから構成され、それを基に4ステージで構成される動作に沿って命令の割り当てを行う。

6.1.1 スコアボードを構成するステータス

スコアボードは、命令の状態を評価する以下の3項目で構成される管理テーブルである。

1. 命令の状態：命令が、発行・実行中・完了の内、どの状態にあるかを評価する。
2. オペランドの使用状態：各オペランドが割り当て可能かどうかを評価する。
3. 関数ユニットの状態：各関数ユニットの使用状況が利用可能かどうかを評価する。

6.2 動作項目

スコアボーディング方式における命令の実行は、主に以下の4ステージに沿って進められる。

1. 発行 (Issue)：命令が解釈され、スコアボードに登録される。命令のオペランドが利用可能な場合は、関数ユニットに送られる。
2. 読み出し (Read Operands)：オペランドが利用可能である場合、関数ユニットから読み取られる。
3. 実行 (Excute)：必要なオペランドが揃うまで待機したのち、命令が関数ユニット内で実行される。
4. 書き戻し (Write Back)：実行結果をレジスタに書き込み、スコアボードを更新する。

6.3 特徴

スコアボーディング方式の特徴として、命令の状態に基づいて適切なパイプラインを割り当てることができることから、並列実行が可能になり、実行効率を上げることができ、実装も比較的容易に行うことができる。対して、命令の依存関係が十分に解析できなかった場合、リソース競合やデータ依存を解消できず、処理が遅くなったりデータに損失が生じる可能性がある。

6.4 トマスロのアルゴリズム

トマスロのアルゴリズムとは、リザベーションステーションおよびリオーダーバッファと呼ばれる機構を用いて命令のスケジューリングと実行を管理する手法である。

スコアボーディングでは、依存関係の管理を命令を一時停止することで実現しているのに対して、トマスロのアルゴリズムでは命令を連続的に発行することが可能であることから、スケジューリングに存在していた動的スケジューリング性の問題を克服している。実装には以下の3項目を実現する必要性があり、それを基に3ステージで構成される動作に沿って命令の割り当てを行う。

6.5 トマスロのアルゴリズムを構成する要素

1. 命令列の及ぼす結果がパイプライン化されていないプロセッサと同期されるよう、命令を順序通りに発行する。
2. 汎用レジスタおよびリザベーションステーションレジスタは、実際の値か仮想的な値を格納する。レジスタが利用できない場合は、割り当てられている仮想値を一時的に利用し、計算中の値は仮想値に割り当てられたうえで、演算完了後に仮想レジスタの値が実際の値に変換される。
3. 機能ユニットにおいてはリザベーションステーションを複数スロットに割り当てて使用し、各スロットには、実行に必要な情報を保持したうえで、データがすべて実際の値に置き換わった際に実行を開始する。

6.6 動作項目

1. 発行 (Issue) : 命令が解釈され、利用可能なリザベーションステーションに送られる。リザベーションステーションは命令のオペランドを待機し、関数ユニットが利用可能になるまで待機する。
2. 実行 (Execute) : オペランドが揃った命令から順に、対応する関数ユニットで実行され、実行結果をリザベーションステーションからリオーダーバッファに送る。
3. 書き戻し (Write back) : 実行結果がリオーダーバッファを通じてレジスタに書き戻され、命令を完了とみなす。

6.7 特徴

レジスタ・リネーミングを採用しているため、再利用レジスタに別の実在レジスタを割り当てることで依存を解消し、不必要な順序依存性を省くことができるため、データ依存と名前による依存を分離することができ、不必要な依存性によって利用されていた領域を解放することにより、より多くの命令を同時に実行できるようになり、プロセッサの性能が大幅に向上する。また、命令のスケジューリングをおこなっていることから、アウトオブオーダー実行も可能である。しかし、リザベーションステーションとリオーダーバッファの管理はとても複雑であるため、ハードウェア上での実装が難しく、設計・製造コストも増加するという問題もある。

7 課題7：キャッシュメモリの構成における「ライトスルー」と「ライトバック」に関する課題

7.1 ライトスルーとは

ライトスルーとは、キャッシュメモリの構成におけるキャッシュ・メモリ間のデータ書き込み手法の1種である。ライトスルー方式は、データの書き込みを行う際にキャッシュとメモリ（主記憶装置）の両方に同時に書き込みを行う手法であり、常にキャッシュとメモリのデータが一致することから、外的要因によるデータの喪失を抑えられ、システムの一貫性を担保することができる。また、メモリの整合性を取りやすいことから、キャッシュの設計が容易であるという特徴もある。一方、キャッシュと比べて低速であるメモリへの書き込みが頻繁に行われるため、処理速度が遅くなり、バスにおけるデータ線の使用率が高くなるという問題点がある。

7.2 ライトバックとは

ライトバックとは、キャッシュ・メモリ間のデータ書き込み手法の1種である。ライトバック方式では、データ書き込みが生じた際に一旦キャッシュへの書き込みのみで書き込みを終了し、処理に空きが生じたタイミングでキャッシュからメインメモリへデータの書き込みを行うため、プロセッサからメモリへの書き込みが即座には行われない。キャッシュラインの置き換えなどが生じる際にメモリ側へデータが書き込まれるため、書き込み動作がキャッシュ内で完結し、書き込み速度が高速であるように見せかけることができ、また、バスの使用率も低く抑えることができる。しかし、キャッシュとメモリのデータが一致しないタイミングが多数生じることから、データの喪失などの問題が生じる可能性があり、特にマルチプロセッシングを行う際には、より複雑な設計が必要になり、管理するハードウェア数の増加や、コストの増加に直結する可能性がある。

7.3 考察

以上のように、ライトスルー・ライトバックに関する内容は、キャッシュメモリの構成において処理速度を大きく左右する項目であるが、指定教科書において、あまり深くは言及されていない。このように重要な項目に関する言及が省かれている理由として私は、教科書において重点に置かれているポイントや、教科書において求められている内容のレベルが関係している可能性がある。この指定教科書において主に扱われている内容は、コンピュータアーキテクチャを扱う上で必要不可欠な基礎的な内容を重点に置いて記述項目が決められている傾向があり、ライトスルー・ライトバックは、キャッシュメモリのパフォーマンスに関する重要なトピックではあるものの、本項目のような詳細な管理ポリシーなどに関する項目はより高度な内容とみなされている可能性があり、基礎を扱うことを目的としている指定教科書においては、名称のみの言及に抑えられている可能性がある。ただし、本項目の内容はキャッシュの構成・管理においてパフォーマンスに直結する重要な項目であることから、用語の説明と簡単な実行速度の比較などの内容は教科書に記述する必要があるのではないかと考えた。

8 課題 8：並列処理アーキテクチャのヘテロジニアスマルチプロセッシングの例を、授業で紹介した例以外について紹介してください。

ヘテロジニアスマルチプロセッシング (HMP) とは、異なる種類のプロセッサコアを使用して、タスクに対してより適切なコアを割り当てることにより効率的に処理を行う技術である。異なる特性を持つプロセッサコアを同じチップ上に配置し、各コアの特性にあったタスクを割り当てることで、処理速度の向上だけでなく、電力効率の向上などに生かすこともできる。動作の特徴としては、高性能コアと低消費電力コアを組み合わせた異種コアを採用し、動的に適切なタスクを割り当てていくことで、高負荷なタスクを高性能コアに、低負荷のタスクを低消費電力コアに割り当てることで、全体を通してバランスの取れた性能と省電力を実現することができる。現代ではスマートフォンの処理性能の向上やバッテリー寿命をより長く持たせるために採用されている。

8.1 ヘテロジニアスマルチプロセッシングの例

8.1.1 Cell BroadBand Engine

Cell Broadband Engine とは、ソニー、東芝、IBM によって共同開発されたヘテロジニアスマルチプロセッサの一つで、ヘテロジニアスマルチコアを採用した初期の 64bitRISC マイクロプロセッサである。このプロセッサは、当時の映像処理や科学技術計算などといった特に高い計算能力を要求されるアプリケーションに向けて設計され、シングルスレッド性能を維持しつつマルチスレッド性能も向上することを目的として採用された。Cell Broadband Engine は、PPE,SPE,EIB の 3つの要素により構成され、それぞれ異なる役割を担っている。

PPE(Power Processing Element)：汎用プロセッサとしての役割を果たし、オペレーティングシステムの管理や全体の制御などを行う。PowerPC アーキテクチャを主体とした、シングルスレッドプロセッサ。SPE(Synergistic Processing Element)：特定の演算タスクに特化した形で設計されたコア。ベクトル演算やマルチメディア処理などに優れている。構成は、効率的なコアを最大 8 つ搭載したものとなっており、PPE から割り当てられたタスクを並列的に処理する。

EIB(Element Interconnect Bus)：PPE・SPE 間のデータ通信を行う高速バスである。各コアが効率的にデータの共有を行えるようにすることができる。

Cell Broadband Engine は PlayStation 3 や当時の Roadrunner などのスーパーコンピュータに搭載され、グラフィックス処理や物理シミュレーション、科学技術計算などに使用された。

8.2 その他の例

Cell Broadband Engine の他には、Qualcomm が開発した Snapdragon 800 シリーズや NVIDIA Tegra X1 などにヘテロジニアスマルチプロセッシングは採用され、現代では主に AI によるシミュレーションやグラフィックス処理に使用されている。

9 課題9：授業で紹介した Google TPU 以外の DSA を紹介してください。その際には「DSA のガイドラインにどこまで沿っているか？」の観点からの見解も含めること。

9.1 NVIDIA CUDA コア

CUDA コアとは、NVIDIA により開発されている GPU に搭載されている処理装置の総称であり、映像処理および並列計算に特化したハードウェアとなっている。また、CUDA プラットフォームと呼ばれる NVIDIA により提供されている並列コンピューティングプラットフォームを使用することで、深層学習などといった分野にも利用することができるようになっている。DSA のガイドラインに沿って CUDA コアを評価すると以下のようになる。

表 5: NVIDIA CUDA コアの評価

DSA ガイドライン	CUDA
・設計対象	GPU アーキテクチャ
1. 最適化されたメモリ構成	L1 cache / 192KB ,L2 cache / 40MB
2. カスタマイズ可能性	ハードウェアにより制限
3. 並列処理	6912 個の CUDA コアによる並列処理、SIMT
4. データパスの最適化・高効率性	8bit,16bit,32bit,64bit の演算
5. 専門的なソフトウェアサポート	CUDA Toolkit,cuDNN,TensorRT,NCCL
6. 演算ユニットの大型化	6912 CUDA コア + 432 Tensor コア

これらの内容から、CUDA コアは、特に大規模な並列処理能力と高効率データパス、そして専門的なソフトウェアサポートにおいて DSA のガイドラインに適合している。また、メモリ構成の高効率化や演算ユニットの大型化にも対応している。

一方で、基本的には GPU アーキテクチャ側のドメインは固定されていることから、ソフトウェアレベルでの汎用性は多少あるものの、ハードウェアレベルでのカスタマイズ可能性は制限されている。

9.2 Xilinx FPGA

FPGA とは、設計者がその場で論理回路の構成をプログラムすることができる集積回路デバイスである。その特徴としては通常の集積回路とは違い、Verilog 等のハードウェア記述用言語を使用してプログラムを構成することで、特定の領域に特化した回路を必要に応じて再構成することができる。

プログラムに応じてハードウェア内部の回路を組み替えることで、映像出力から計算まで、幅広い分野に特化した集積回路を必要に応じて作り出すことができるようになっている。

DSA のガイドラインに沿って FPGA を評価すると、以下のようになる。

表 6: Xilinx FPGA の評価

DSA ガイドライン	FPGA
・設計対象	再構成可能なハードウェアプラットフォーム
1. 最適化されたメモリ構成	Block RAM 75.9Mb ,UltraRAM 90Mib (VU19PD)
2. カスタマイズ可能性	すべてのロジックブロック及び I/O ピンをプログラム可能
3. 並列処理	数百万の論理素子による並列処理
4. データパスの最適化・高効率性	8bit,16bit,32bit のデータ幅を選択可能
5. 専門的なソフトウェアサポート	Vivado, Vitis, HLS
6. 演算ユニットの大型化	3840 個の DSP ユニット

これらの内容から、Xilinx FPGA は、メモリ構成やカスタマイズ可能性、最適化という面において DSA のガイドラインに適合しており、深層学習などといった特定のアプリケーションに最適化された設計が可能であることが分かる。また、並列処理性能や専門的なソフトウェアサポートも優れている一方で、汎用性重視で設計されていることから、演算ユニットの大型化が専用 ASIC に及んでいないという欠点も存在している。